the highest. The probability of each token for x and $x+v$ can be computed by all the bs-values:

$$p(w|x) = \frac{exp(bs_w^x)}{exp(bs_1^x) + ... + exp(bs_B^x)} \quad (10)$$

$$p(w|x+v) = \frac{exp(bs_w^{x+v})}{exp(bs_1^{x+v}) + ... + exp(bs_B^{x+v})} \quad (11)$$

where bs-value bs_w^{x+v} is equal to $bs_w^x + bs_w^v$:

$$bs(x+v) = bs(x) + bs(v) \quad (12)$$

Although the probability change is nonlinear, the change on each token's bs-value is linear. In order to analyze which components of v is important, we design several $bs(x)$ and $bs(v)$ and compute the distribution change. Assume there are four tokens in vocabulary space, and $bs(x) = [1, 2, 3, 4]$. The probability distribution of x is $[0.03, 0.09, 0.24, 0.64]$. We design several v and compute the probability distribution of $p(x+v)$. The details are shown in Table 1.

$bs(v)$	$bs(x+v)$	$p(x+v)$
$[1,1,1,3]$	$[2,3,4,7]$	$[0.01,0.02,0.05,0.93]$
$[3,1,1,1]$	$[4,3,4,5]$	$[0.20,0.07,0.20,0.53]$
$[6,4,4,4]$	$[7,6,7,8]$	$[0.20,0.07,0.20,0.53]$
$[6,2,2,2]$	$[7,4,5,6]$	$[0.64,0.03,0.09,0.23]$
$-[6,2,2,2]$	$[-5,0,1,2]$	$[0.00,0.09,0.24,0.67]$

Table 1: Probability distribution of $p(x+v)$.

Existing studies (Geva et al., 2022; Lee et al., 2024) state that $p(w|x+v) \propto exp(e_w \cdot v)$. However, based on the examples provided in Table 1, not only the bs-value of token w, but also the bs-values of all the tokens affect the probability. For example, $bs(v) = [3, 1, 1, 1]$ and $[6, 4, 4, 4]$ result in the same distribution, although the bs-value of each token is enlarged.

An intuitive observation is that v aids in magnifying the token with the largest bs-value. For instance, $[1, 1, 1, 3]$ is conducive to increasing the probability of the last token, and $[3, 1, 1, 1]$ can amplify the probability of the first token. This observation may elucidate why many neurons exhibit human-interpretable concepts when projected into the vocabulary space. Given that the vocabulary size B is typically large (often exceeding 30,000), probabilities of tokens with the largest bs-values are likely to be augmented.

Another significant finding is that both the coefficient score and the neuron's bs-values play substantial roles. Compared with $[3, 1, 1, 1]$, $[6, 2, 2, 2]$ can can both amplify and diminish the probability change of $[3, 1, 1, 1]$. The probability increase of the first token is magnified, while the decrease in probability of the last token is more pronounced. When the coefficient score's sign is changed (e.g. $-[6, 2, 2, 2]$), the effect on the first token's probability changes from increasing to decreasing.

3.3 Importance Score for "Value Neurons"

Based on the analysis in Section 3.2, an intuitive importance score of a neuron mv is $|m| \times |1/rank(w)|$, where m is the coefficient score and $rank(w)$ denotes the ranking of the final token when projecting v into vocabulary space. Another intuitive importance score is calculating the probability $p(w|mv)$ on token w. If these scores are large, v will contain much information of w.

However, these methods have two potential problems. On one hand, they only consider the effect of v, overlooking the varying importance of v under different x conditions. On the other hand, we usually hope to analyze the importance of different modules' combination. Therefore, it is better that the importance score Imp satisfies $Imp(x+v) \approx Imp(x) + Imp(v)$.

To address these problems, we design log probability increase as importance score for both layer-level and neuron-level vectors. If v^l is a vector in lth attention layer, the importance score of v^l is:

$$Imp(v^l) = log(p(w|v^l + h^{l-1})) - log(p(w|h^{l-1})) \quad (13)$$

where the probability of each vector is computed by multiplying the vector with E_u (see Eq.2). If v^l is a vector in lth FFN layer, we compute the importance score by replacing h^{l-1} as $h^{l-1} + A^l$ in Eq.13. In Eq.13, v^l is not the only element controlling the importance score. Also, it is convenient for analyzing the combination of different modules.

3.4 Importance Score for "Query Neurons"

As discussed in Section 3.3, the proposed attribution methods can effectively identify the "value neurons" containing crucial information for the final prediction. However, in addition to these "value neurons", there exist "query neurons" that aid in activating these neurons, even if they may not directly contain information about w. In this section, we propose a static method to identify these "query

neurons" based on Eq.1, Eq.5, and Eq.6. Since the $fc2$ vectors do not change, the coefficient scores are the only varying element in different cases. For each "value neuron", we can compute the inner product between its subkey (see Eq.6) and each neuron/subvector within the residual output (see Eq.1). Despite the presence of a nonlinear function σ for computing the coefficient score, it usually does not affect the relative value between different neurons/subvectors. Therefore, if a "query" neuron/subvector exhibits a larger inner product with the subkey compared to another one, it is more helpful for activating the "value neuron".

4 Experiments

In this section, we compare our neuron-level attribution method with seven other methods in Section 4.1. Then we analyze the localization of six types of knowledge using our method in Section 4.2.

4.1 Comparison of Attribution Methods

We compare the proposed method in Eq.13 with seven other methods. For each sentence, we apply every method to identify top10 FFN neurons, and evaluate the attributed neurons using three metrics.

Dataset. We extract query-answer pairs with six types of answer tokens (language, capital, country, color, number, month) from TriviaQA (Joshi et al., 2017). To explore the mechanism of knowledge storage, we extract all the sentences where the correct token ranks within the top10 predictions and higher than other words in the same knowledge in GPT2-large (Radford et al., 2019) and Llama-7B (Touvron et al., 2023). There are 1,350 sentences for GPT2-large and 3,141 sentences for Llama-7B.

Models. To compare the differences between large and small models in terms of knowledge storage, we conduct experiments on Llama-7B and GPT2-large. Llama-7B consists of 32 layers, with each attention layer comprising 32 heads, each head containing 128 neurons and each FFN layer containing 11,008 neurons. GPT2-large has 36 layers with 20 heads per attention layer, 64 neurons per head, and 5,120 neurons per FFN layer.

Attribution methods. We compare our method with seven static methods. We use each method to attribute the FFN neurons with top10 scores for the correct knowledge token w. Similar to Eq.5, each neuron mv is the product of the coefficient score m and $fc2$ vector v^l. Here are the methods:

a) (proposed method) log probability increase: $log(p(w|mv^l + A^l + h^{l-1})) - log(p(w|A^l + h^{l-1}))$
b) log probability: $log(p(w|mv^l))$, attributing the same neurons with $p(w|mv^l)$. This is similar to direct logit attribution (DLA) in Wang et al. (2022).
c) probability increase: $p(w|mv^l + A^l + h^{l-1}) - p(w|A^l + h^{l-1})$
d) norm: $|v^l|$
e) coefficient score: $|m|$
f) ranking in vocabulary space: $1/rank(w)$
g) $|m| \times |v^l|$, introduced in Geva et al. (2022).
h) $|m| \times 1/rank(w)$

Metrics. We devise three metrics to evaluate the attribution methods. After attributing the top10 FFN neurons by each method, we intervene on these neurons by setting the top10 neurons' parameters to zero. Subsequently, we rerun the model and compute the Mean Reciprocal Rank (MRR) score of the correct token w, the probability of w (prob), and the log probability of w (logp). An attribution method is considered superior when it exhibits greater decreases in these metrics.

	GPT2-large			Llama-7B		
	MRR	prob	logp	MRR	prob	logp
o)	0.361	7.1	-3.15	0.551	21.5	-2.24
a)	**0.201**	**3.4**	**-4.06**	**0.312**	**9.2**	**-3.91**
b)	0.214	3.6	-3.91	0.339	10.8	-3.35
c)	0.219	3.7	-3.92	0.345	10.0	-3.57
d)	0.363	7.1	-3.14	0.549	21.3	-2.25
e)	0.439	8.6	-3.10	0.529	22.9	-2.35
f)	0.306	5.8	-3.40	0.493	18.1	-2.49
g)	0.394	8.1	-3.06	0.523	22.6	-2.39
h)	0.232	4.0	-3.80	0.389	13.0	-3.06

Table 2: Results of attribution methods on two models.

Results and analysis. The results of the original model (first line) and eight attribution methods are shown in Table 1. In comparison with the other seven methods, our attribution method (second line) attributes more important neurons, resulting in the most significant reduction across all metrics in both GPT2 and Llama. Specifically, when only intervening ten FFN neurons, the probability of the correct knowledge token reduces from 7.1% to 3.4% in GPT2, and from 21.5% to 9.2% in Llama-7B. This indicates that there are several neurons storing much important information for knowledge storage, and our method can locate these neurons.

The attribution methods of norm v^l (d) and $m \times |v^l|$ (g) are not useful, which indicates the norm

of neurons is not important for attribution. Using $|m| \times 1/rank(w)$ (h) has good results, which is better than $1/rank(w)$ (f). The ranking of tokens in vocabulary space for projected neurons is a good saliency score, and the coefficient score can enlarge the distribution change, aligning our analysis in Section 3.2. Only using coefficient score (e) is not helpful for attribution. The role of coefficient score is to enhance the probability change caused by the neuron, but whether the neuron is useful for the selected token depends on the neuron itself. There are other tokens competing with the correct knowledge token, so the neurons with large coefficient scores may be related to these tokens.

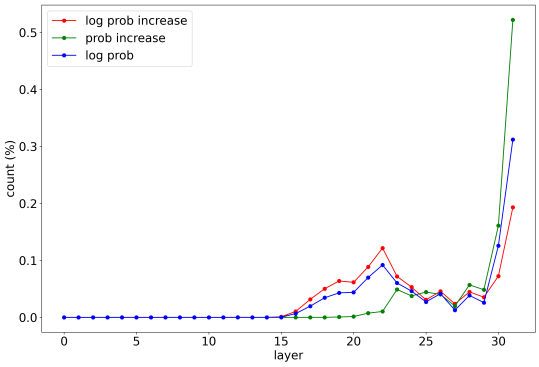

Figure 2: Neuron distribution on all layers in Llama-7B.

Compared to log probability (b), employing log probability increase (a) can attribute more important neurons. This aligns with the analysis in Section 3.2 and 3.3: not only neuron v, but also x affects $p(w|x+v)-p(w|x)$. Compared with probability increase (c), log probability increase achieves better results. We analyze the distribution of neurons across all layers in Llama attributed by log probability increase, log probability, and probability increase, as depicted in Figure 2. GPT2 has similar results, detailed in Appendix A. The neurons attributed by probability increase are on deepest layers ($23th - 31th$), while other two methods can attribute neurons among $17th$ to $31th$ layers.

To delve into the reason of this phenomenon, we analyze the difference of importance score when adding the same vector v on different x. As discussed in Eq.13, the importance score of v is computed by $log(p(w|x+v)) - log(p(w|x))$. Therefore, the importance score is related to the curve of $F(a) = log(p(w|a))$. To analyze this curve, we compute the final vector h_T^L and the $0th$ layer input vector h_T^0 on each sentence, and divide $h_T^L - h_T^0$ into 61 segments, where each segment is $Seg_S = h_T^0 + S(h_T^L - h_T^0)/60$ (S is the segment index from 0 to 60). Then we compute the probability $p(w|Seg_S)$ and log probability $log(p(w|Seg_S))$ at each segment index for every sentence. The average score on Llama-7B is shown in Figure 3.

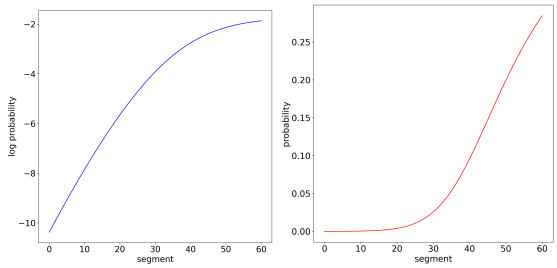

Figure 3: Curves of log probability increase (left) and probability increase (right) on Llama-7B.

The curve of log probability increase exhibits an approximately linear shape from 0 to 40 segments, while the curve of probability increase shows a linear trend from 40 to 60 segments. This observation elucidates the findings in Figure 2: employing probability increase is more inclined to attribute neurons in the deepest layers, whereas log probability increase tends to attribute neurons in medium-deep layers. Despite the slower slope of the log probability increase curve in very deep layers, it still effectively attributes neurons in very deep layers (as depicted in Figure 2). This maybe because neurons in very deep layers contain substantial information, and even when the importance score decreases, it remains relatively large. In later sections, we use log probability increase as importance score for exploration, as this method can identify the important neurons in both medium-deep layers and very deep layers, and its experimental results are the best. Nevertheless, reproducing the importance of the deepest layers may be a prospective avenue for developing improved attribution methods.

4.2 Exploration on Different Knowledge

We take log probability increase as importance score, and analyze six types of knowledge: language (lang), capital (capi), country (cnty), color (col), number (num), and month (mon). We evaluate the knowledge storage in attention and FFN layers at layer-level, head-level, and neuron-level. We also compute the storage of query layers and query neurons.

Layer-level knowledge storage. We compute the sum of importance score of each attention and FFN layer in GPT2 (G-A, G-F) and Llama (L-A, L-F), shown in Table 3. Also, we calculate the top10